

Module 17: STL Algorithms & Lambdas

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

A genuinely fun, high-leverage module: students replace hand-written loops they've written a dozen times (find a max, count matches, convert values) with one-line algorithm calls. Lambdas are the enabling idea - introduce them as 'a function written exactly where you need it,' not as an abstract new syntax category.

Suggested Duration: 75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Use `std::sort`, `std::max_element/min_element`, and `std::find` on any container.
- Write a lambda expression, including its capture list, parameters, and body.
- Use `std::count_if` with a lambda predicate to count matches.
- Use `std::transform` with a lambda to build a new sequence from an old one.
- Capture a variable from the surrounding scope by value or by reference, and explain the difference.

Prerequisites

- Module 16 (STL containers, iterators - every algorithm here operates on a `[begin, end)` iterator range).

Materials & Instructor Prep

- `17-stl-algorithms-lambdas/examples/03_lambdas_and_count_if.cpp` preloaded - the single best example for the 'this replaces a loop I've written many times' framing.

Key Vocabulary

Term	Definition
Algorithm (STL sense)	A ready-made, tested function from <algorithm> operating over a [begin, end) iterator range - sort, find, count_if, transform, and others.
Lambda	A small, unnamed function defined inline, exactly where it's needed.
Capture list ([])	The part of a lambda specifying which surrounding variables it can use, and whether by value or by reference.
Predicate	A function (often a lambda) returning true/false, used by an algorithm to decide whether each element qualifies.

Lesson Outline & Instructor Talking Points

1. sort, max_element, find - familiar tasks, new tools (20 min)

- Before showing `std::sort`, ask: 'how would you have sorted a vector by hand, with the tools from Module 4?' Let them describe (or even briefly sketch) a bubble/selection sort loop - then reveal `std::sort(scores.begin(), scores.end())` does the same job in one line.
- Run `examples/01_sort_and_find.cpp` and `examples/02_max_min_element.cpp`. Emphasize the [begin, end) range pattern is now used by EVERY algorithm in this header - 'learn this pattern once, it's the same everywhere.'
- Stress that `max_element/min_element` return an ITERATOR, not the value - dereference with `*` before using it, exactly like Module 16's iterators.

2. Lambdas - a function written on the spot (20 min)

- Write a plain named function `bool isPassing(int score) { return score >= 50; }` first - something totally familiar. THEN rewrite it as a lambda: `auto isPassing = [](int score) { return score >= 50; };`
- Walk through each part of the lambda syntax by pointing directly at the corresponding part of the named function it replaced: `[]` (no name needed), `(int score)` (same parameters), `{ return ...; }` (same body).

Instructor note: *Building the lambda AS a rewrite of a named function the class already trusts is far more effective than introducing lambda syntax cold.*

3. count_if with a lambda predicate (15 min)

- Run `examples/03_lambdas_and_count_if.cpp`. Show the lambda passed BOTH as a stored variable and written directly inline with no name at all - let students see both styles and note the inline style is far more common in real code once the pattern feels natural.

4. transform, and capturing variables (20 min)

- Run examples/04_transform_and_capture.cpp. Point out the destination vector must already be sized (voltages(rawAdcCounts.size())) - transform WRITES into existing slots, it doesn't push_back.
- Introduce capture explicitly: [threshold] lets the lambda read a variable from outside itself. Show [=] and [&] briefly, but state the course's preference: name exactly what you capture rather than capturing everything out of habit.

Common Student Misconceptions

Misconception	How to Address It
"max_element/min_element return the actual maximum/minimum VALUE."	They return an ITERATOR pointing at that element - dereference with * to get the value, exactly like any other iterator.
"std::transform grows the destination container as needed, like push_back would."	It writes into EXISTING elements of the destination - the destination must already be sized correctly beforehand.
"[=] or [&] is always the simplest, safest choice for a lambda's capture list."	Naming exactly what's captured (e.g. [threshold]) is clearer about intent and safer against accidentally depending on more than intended - prefer it by default.

Exercises & Module Project

Exercise: 17-stl-algorithms-lambdas/exercises/exercise1.cpp - sort/max/min/count_if/transform applied to a temperature series. Project (three tracks): Generic - Exam Score Analyzer; EE - Waveform Analyzer (ADC-to-voltage conversion, peak detection, over-voltage counting); Biomedical - Vital Sign Analyzer (abnormal-reading counting, a risk-score transform).

Facilitation guidance: All three tracks chain the SAME four algorithms (sort, max/min_element, count_if, transform) in slightly different domain dress - if a student is stuck, have them name which of the four operations they're trying to do before worrying about the domain-specific formula.

Discussion & Comprehension Check Questions

1. Rewrite (verbally or on the board) a hand-written loop that counts values above a threshold as a count_if call. What changes, and what stays conceptually the same?

Guidance: The looping/counting mechanism is now hidden inside count_if; what the STUDENT writes is only the per-element test (the lambda) - the logic of 'what counts' stays theirs, the

mechanics of iterating don't.

2. Why must a lambda's capture list explicitly name threshold to use it, instead of just referencing the variable directly?

Guidance: A lambda is its own small function - like any function, it can only see variables passed to it (as parameters) or explicitly captured from its surrounding scope; capture is how a lambda opts into 'seeing' an outside variable.

Assessment Ideas

- Given a plain English task ('find how many readings exceed 100'), have students write just the `count_if` call with an inline lambda, no surrounding program.

Answer Key Notes

See [17-stl-algorithms-lambdas/exercises/solutions/exercise1_solution.cpp](#).

Wrap-Up & Bridge to Next Module

Bridge forward: "Your `average()`/Statistics-style functions have been rewritten once per type so far. Next module shows you how to write that logic exactly once, for ANY type - templates, the same idea behind `vector<T>` and `map<K,V>` themselves."

Instructor Tips

- This module tends to run FASTER than expected once the lambda-as-rewrite framing lands - have the project starter files open and ready in case there's extra time to start it in class.